

ZAP · SECURITY REVIEW

Zap
node sale.

A security review of node sale.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Zap	node sale	Guardian	October 1 – October 10, 2024	October 17, 2024

§ 00

Contents.

59 pages — overview, confidence ranking, scope, 46 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 38 findings	P. 10
	Remediation Review · 8 findings	P. 50
§ 05	Disclaimer	P. 58
§ 06	About Guardian	P. 59

§ 01

Overview.

Guardian reviewed node sale across 2 review rounds. The final report includes 8 Critical, 13 High, 8 Medium, 17 Low findings.

AUDIT FIRM	REPOSITORY
Guardian	Lithium-Ventures/zap-node-sale
CLIENT	METHODOLOGY
Zap	Smart contract · Manual · Main Review · Remediation Review
PROJECT	MAIN REVIEW COMMIT
node sale	50066b0e6332
PREPARED BY	
Guardian	
AUDIT WINDOW	
October 1 – October 10, 2024	
FINAL REPORT	
October 17, 2024	

1.1 Confidence Ranking

1 / 5

Very Low Confidence

Guardian assigns a Confidence Ranking of 1 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.



1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
■ Critical	8	0	1	0	7
■ High	13	0	2	0	11
■ Medium	8	0	5	0	3
■ Low	17	0	6	0	11
■ Informational	0	0	0	0	0
All severities	46	0	14	0	32

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence ASSIGNED	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed node sale in `Lithium-Ventures/zap-node-sale`. The scope includes 948 source lines across 4 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
contracts/ZapToken.sol	17	24
contracts/ZapStaking.sol	320	448
contracts/ZapRewards.sol	293	430
contracts/VaultVesting.sol	318	476

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
C-01	DOS of importUserData	DOS	CRITICAL	RESOLVED	p. 10
C-02	DOS of Bonus Weight Calculation in VaultVesting	DOS	CRITICAL	RESOLVED	p. 11
C-03	Early Unstake Fee Paid By VaultVesting	Logical Error	CRITICAL	RESOLVED	p. 12
C-04	Unaccounted User Staking	Logical Error	CRITICAL	RESOLVED	p. 13
C-05	Lost Rewards Over Multiple Claims	Logical Error	CRITICAL	RESOLVED	p. 14
C-06	Invalid Tiers Lead To DOS Of Rewards Accrual	DOS	CRITICAL	RESOLVED	p. 15
C-07	Lack Of Amount Validation Leads To DOS	DOS	CRITICAL	RESOLVED	p. 16
H-01	Unstake Fee Bypassed	Logical Error	HIGH	RESOLVED	p. 17
H-02	endIndex Is Incorrect When The Chonk Is Final	Logical Error	HIGH	RESOLVED	p. 18
H-03	previousStake Not Updated For Unstaking Tokens	Logical Error	HIGH	RESOLVED	p. 19
H-04	Underflow In _secondsToDays	Logical Error	HIGH	RESOLVED	p. 20
H-05	User Can Influence Daily Vesting Update	Logical Error	HIGH	RESOLVED	p. 21
H-06	Lost Rewards When Unstake	Logical Error	HIGH	ACKNOWLEDGED	p. 22
H-07	User Can Influence Distribution In ZapRewards	Logical Error	HIGH	RESOLVED	p. 23

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
H-08	Incorrect Forecasted Rewards	Logical Error	HIGH	RESOLVED	p. 24
H-09	existingLockedTokensForStake Wrong Denominator	Logical Error	HIGH	RESOLVED	p. 26
H-10	Not earning rewards after vesting ends	DOS	HIGH	RESOLVED	p. 27
H-11	Continue over require in save	DOS	HIGH	RESOLVED	p. 28
H-12	Early Unstake Fee Calculation Could Underflow	DOS	HIGH	RESOLVED	p. 29
M-01	Missing Recovery Functions	Logical Error	MEDIUM	RESOLVED	p. 30
M-02	Less rewards for expired stakes	Logical Error	MEDIUM	ACKNOWLEDGED	p. 31
M-03	Off by One Error In vestingPeriodHasNotElapsed	Off By One	MEDIUM	RESOLVED	p. 32
M-04	Restaking Can Be Gamed For Higher Rewards	Logical Error	MEDIUM	RESOLVED	p. 33
L-01	Inconsistency Between startingIndexOfChonks	Warning	LOW	RESOLVED	p. 34
L-02	Gas Wastage In Loop	Gas Optimization	LOW	RESOLVED	p. 35
L-03	Wrong VaultVesting iterations count	Logical Error	LOW	ACKNOWLEDGED	p. 36
L-04	Uninitialized Treasury Account In ZapStaking	Warning	LOW	RESOLVED	p. 37
L-05	Unused State Variables	Superfluous Code	LOW	RESOLVED	p. 38
L-06	Incorrect Parameter In Unstaked Event	Events	LOW	RESOLVED	p. 39
L-07	Forecast Doublecounting Current Day	Warning	LOW	ACKNOWLEDGED	p. 40
L-08	Updating Wrong Variable For Token Staker Rewards	Warning	LOW	RESOLVED	p. 41

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-09	Vault staking incentives	Warning	LOW	ACKNOWLEDGED	p. 42
L-10	Multiply Before Dividing	Precision	LOW	RESOLVED	p. 43
L-11	Daily function may be executed more	Validation	LOW	ACKNOWLEDGED	p. 44
L-12	Incorrect Tokens Emitted On Certain Days	Logical Error	LOW	RESOLVED	p. 45
L-13	preTGE ratio is the same for all users	Warning	LOW	RESOLVED	p. 46
L-14	Potentially Unexpected Returning Staker Calculation	Logical Error	LOW	RESOLVED	p. 48
L-15	Import Can Be Finished Multiple Times	Validation	LOW	RESOLVED	p. 49

REMEDIATION REVIEW

C-01	Underflow _createNewExpiryTimeForAutoRelock()	—	CRITICAL	ACKNOWLEDGED	p. 50
H-01	Wrong standardPoolNegativeMultiplier Applied	—	HIGH	ACKNOWLEDGED	p. 51
M-01	Wrong halfway time calculation	—	MEDIUM	ACKNOWLEDGED	p. 52
M-02	Halfway calculations ignore autoRelock	—	MEDIUM	ACKNOWLEDGED	p. 53
M-03	Wrong autoRelock unstake fee	—	MEDIUM	ACKNOWLEDGED	p. 54
M-04	recoverTokensAfterVesting off by one	—	MEDIUM	ACKNOWLEDGED	p. 55
L-01	Malicious token recovery	—	LOW	ACKNOWLEDGED	p. 56
L-02	Unstaking fees affects rewards	—	LOW	ACKNOWLEDGED	p. 57

CRITICAL **C-01**

RESOLVED

DOS of importUserData

CATEGORY	DOS
LOCATION	VaultVesting.sol
REVIEW	Main Review

DESCRIPTION

An attacker can DOS the import of user data by front-running with `updateVaultsInfoForUser`.

This increases the `nextAccountID` and causes a mismatch in `accountId` and `indexGiven` when `importUserData` is called by admin.

The attack can be performed repeatedly at low cost (only gas cost incurred) and delays the commencement of vesting.

RECOMMENDATION

Consider allowing `updateVaultsInfoForUser` to be called only after import is complete.

RESOLUTION

Status: Resolved.

<https://github.com/Lithium-Ventures/zap-node-sale/pull/6>

CRITICAL **C-02**

RESOLVED

DOS of Bonus Weight Calculation in VaultVesting

CATEGORY	DOS
LOCATION	VaultVesting.sol: L298
REVIEW	Main Review

DESCRIPTION

`calculateTotalBonusWeightingForChonk` is an important function used to calculate the daily total bonus weight of users.

The function requires that `nextAccountId` in `ZapAccount` and `VaultVesting` are equivalent:

```
require(vaultCountPlusOne == nextAccountId + 2)
```

This assumes that master accounts do not have a `VaultVesting` id. However, anyone can call `updateVaultsInfoForUser` on either of the two master accounts which would increase `VaultVesting`'s `nextAccountId` and cause this check to always fail.

RECOMMENDATION

Prevent `updateVaultsInfoForUser` from being called on master accounts:

```
require(accountId != 1 && accountId != 2, "master accounts");
```

RESOLUTION

Status: Resolved.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/23a738bfc78f9f613871986ec3af58e0b1386e32>

CRITICAL **C-03**

RESOLVED

Early Unstake Fee Paid By VaultVesting

CATEGORY	Logical Error
LOCATION	ZapStaking.sol
REVIEW	Main Review

DESCRIPTION

When a user unstakes their vaults early, they will have to pay an unstake fee. However, that unstake fee is simply paid by having the VaultVesting contract transfer the fee from its balance of ZAP tokens to the treasury with `vaultVesting.payEarlyUnstakingFee(fee);`

Consequently, a user can continuously stake and unstake to drain the VaultVesting contract out of its ZAP balance. This is especially detrimental since these ZAP tokens are used for the claiming vested tokens. Even if the protocol replenished the VaultVesting contract, a malicious user can continuously do it at the expense of cheap gas.

RECOMMENDATION

Consider requiring vault holders to pay a fee in ZAP tokens.

RESOLUTION

Status: Resolved.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/fa6fae5336fdc7e2a31e9675cf50abfa24a89ad6>

CRITICAL C-04

RESOLVED

Unaccounted User Staking

CATEGORY	Logical Error
LOCATION	ZapRewards::L109
REVIEW	Main Review

DESCRIPTION

When `_calculateTotalTokenStakerTokens()` is called, it will call `getUserTokenStakes()` to determine how many tiers the user is staked in. `getUserTokenStakes()` will return an array with the exact amount of tiers and then iterate through the array's length.

During the iteration, it will call `getTokenStake()` based off what iteration it is on and not what tier it represents. If a user stakes in tier 1 and tier 3, it will grab the stake data for tier 1 and tier 2. This will exclude the stake data from the tier they are actually staked in, which in this case is tier 3. This will lead to a user losing out on tokens that they earned.

RECOMMENDATION

Loop through each tier and if the user does not have any tokens staked in that tier then `continue`.

RESOLUTION

Status: Resolved.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/376ee1998e4fffb6c64176d5e8d0a69757955c54c>

CRITICAL C-05

RESOLVED

Lost Rewards Over Multiple Claims

CATEGORY Logical Error
LOCATION VaultVesting.sol: 226
REVIEW Main Review

DESCRIPTION

In `claimVaults`, users will claim less tokens over multiple claims.

This is due to `_calculateUnlockedTokensForClaimedVaults` using `vaultsHeld` as the denominator. Consider this example:

- Alice has 100 locked tokens and 4 vaults
- Alice claims 2 vaults: $(2/4) * 100 = 50$, new locked is 50
- Alice claims 1 vault: $(1/4) * 50 = 12.5$ tokens, new locked is 37.5
- Alice claims her last vault: $(1/4) * 37.5 = 9.4$ tokens, new locked is 28.1

The remaining amount of Alice's locked tokens cannot be claimed after.

RECOMMENDATION

Adjust the `state.standardPoolTokensLocked` by `(vaultsToClaim / (vaultsHeld - vaultsClaimed))`.

RESOLUTION

Status: Resolved.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/d50749ec78b01d712546e60df0e180c6c92db1e1>

CRITICAL **C-06**

RESOLVED

Invalid Tiers Lead To DOS Of Rewards Accrual

CATEGORY	DOS
LOCATION	ZapStaking.sol
REVIEW	Main Review

DESCRIPTION

Functions `stakeTokens` and `stakeVaults` do not validate that the `tierIndex` is a valid tier. A user could stake for an invalid tier, but still be considered an active staker. When performing `saveCalculatedVaultHolderRewardsToState` function `_calculateEligibleStandardPoolTokensForUser` will be entered since the `vaultStaker` is considered active and a non-zero address is returned.

Since the user does not have stakes in any of the valid tiers, the `totalStandardPoolTokens` returned will just be zero and the following would trigger a revert: `require(standardPoolTokensEligibleForRewards > 0, "ZapRewards: User has no eligible rewards");`

Ultimately, by staking in an inappropriate tier, a user is able to DoS the accrual of standard pool and bonus pool rewards for vault holders.

RECOMMENDATION

Validate that the `tierIndex` is 0-2 when staking.

RESOLUTION

Status: Resolved.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/8830c6165c3371ea0f6258646eac1a0b38a52874>

CRITICAL **C-07**

RESOLVED

Lack Of Amount Validation Leads To DOS

CATEGORY DOS
LOCATION ZapStaking.sol
REVIEW Main Review

DESCRIPTION

Functions `stakeTokens` and `stakeVaults` do not validate that `amountToStake` is valid. A user could stake zero amounts but still be considered an active staker.

When performing `saveCalculatedVaultHolderRewardsToState` function `_calculateEligibleStandardPoolTokensForUser` will be entered since the `vaultStaker` is considered active and a non-zero address is returned.

Since the user does not have staked amount, the `totalStandardPoolTokens` returned will just be zero and the following would trigger a revert: `require(standardPoolTokensEligibleForRewards > 0, "ZapRewards: User has no eligible rewards");`

Ultimately, by staking zero amounts, a user is able to DoS the accrual of standard pool and bonus pool rewards for vault holders.

RECOMMENDATION

Validate that `amountToStake` is greater than zero for vaults and greater than a minimum amount for tokens.

RESOLUTION

Status: Resolved.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/e6bf942deeb02a328379fa4f2430f7c80d7adcc9>

HIGH **H-01**

RESOLVED

Unstake Fee Bypassed

CATEGORY Logical Error
LOCATION ZapStaking.sol
REVIEW Main Review

DESCRIPTION

Users are supposed to be charged a penalty if they unstake early prior to expiration. However, a user can simply stake with `autoRelock` and always avoid the penalty logic, causing no disincentive to unstake early and a loss of fees for the protocol.

RECOMMENDATION

Remove the direct check for `autoRelock` from the `if-statement` that determines if a user has to pay a fee for unstaking early. A potential solution may be: `timeNow < stake.expiresAt || (stake.autoRelock && !isWithinGracePeriod)`

RESOLUTION

Status: Resolved.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/ff22c060e6d969fea777b0342bedd4b7b69b7228>

HIGH **H-02**

RESOLVED

endIndex Is Incorrect When The Chonk Is Final

CATEGORY	Logical Error
LOCATION	ZapRewards.sol
REVIEW	Main Review

DESCRIPTION

The `endIndex` is equal to `amountOfLoopIterations` when it is final chonk. However, this is incorrect because `amountOfLoopIterations` is not the end index; it represents the difference between the start index and the end index.

For example:

```
startingIndexOfChonk : 100,  
sizeOfChonk : 20,  
totalUsersWithStake : 110
```

As a result of this, `isFinal` will be true, `amountOfLoopIterations` will be 10, and `endIndex` will also be 10. The final chonk will not be iterated at all since the loop `for (uint32 i = startingIndexOfChonk; i < endIndex; i++)` will not be executed.

RECOMMENDATION

The `endIndex` should be `amountOfLoopIterations + startingIndexOfChonk` regardless of whether `isFinal` is true or not, since `amountOfLoopIterations` is already updated in the `_prepareChonk` function based on the total number of stakers.

RESOLUTION

Status: Resolved.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/b442ea13bff1089952abedc0ccd73f6b231a6770>

HIGH **H-03**

RESOLVED

previousStake Not Updated For Unstaking Tokens

CATEGORY	Logical Error
LOCATION	ZapStaking.sol: 309-312
REVIEW	Main Review

DESCRIPTION

The two most important parameters in the `TokenStake` struct are `tokensStaked` and `previousStake`. Ideally, these should be updated whenever a new stake or unstake action is performed. However, the `previousStake` variable is not updated in the `unstakeTokens` function.

Since this state variable is used when calculating rewards in the `ZapRewards` contract, incorrect state updates will lead to both user-specific rewards and the total reward weight being calculated incorrectly, leading to loss of rewards for users.

RECOMMENDATION

Update the `stake.previousStake` state variable within the `unstakeTokens` function.

RESOLUTION

Status: Resolved.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/bde7b5817e29af79649289bdac11ea98cd06821e>

HIGH **H-04**

RESOLVED

Underflow In `_secondsToDays`

CATEGORY	Logical Error
LOCATION	ZapStaking.sol
REVIEW	Main Review

DESCRIPTION

ZAP offers an `autoRelock` feature which allows a user to keep earning rewards post-expiration of their stake. Within `_calculateEligibleStandardPoolTokensForUser`, the `_checkIfShouldProcessStakeForReward` internal function will determine that because the stake is `autoRelock`, it is a valid stake for reward processing even if the expiry is passed.

Although the stake is valid and processing is attempted, an underflow will occur in `uint256` `daysRemainingInStake = _secondsToDays(vaultStake.expiresAt - timeNow);` because the expiration has been passed, so `expiresAt < timeNow`. Ultimately, this user (and chunk) will be unable to have vault rewards calculated, and the relock feature does not work as intended.

RECOMMENDATION

Add separate handling for auto-relocking stakes such as an automatic extension of the expiry so underflow does not occur.

RESOLUTION

Status: Resolved.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/2fb08e31cdb095a29a32bba2f21879f7ed4938b2>

HIGH **H-05**

RESOLVED

User Can Influence Daily Vesting Update

CATEGORY Logical Error
LOCATION VaultVesting.sol
REVIEW Main Review

DESCRIPTION

In `calculateTotalBonusWeightingForChonk`, the total bonus pool weight for all users is calculated and stored. This assumes that when `calculateDailyVestingForChonk` is called, that the users' info has not been altered.

A malicious user could increase his share by purchasing more vaults or getting more referrals in between this two-step process. He would then earn more rewards at the expense of other users.

RECOMMENDATION

Consider locking updates to user info in between this two-step process.

RESOLUTION

Status: Resolved.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/cd3ced611c30721121c459fed19e6647a4cf8b62>

HIGH **H-06**

ACKNOWLEDGED

Lost Rewards When Unstake

CATEGORY	Logical Error
LOCATION	ZapRewards
REVIEW	Main Review

DESCRIPTION

It is intended that users still receive rewards based on their previous stake when they unstake after the reward distribution has started, and the `_checkIfShouldProcessStakeForReward` function is used for this purpose.

The `_checkIfShouldProcessStakeForReward` function is called within the `_calculateEligibleStandardPoolTokensForUser` method, using data retrieved from `zapStaking.getUserVaultStakes`.

However, when a vault is unstaked, its `vaultsStaked` value is set to 0, which causes `getUserVaultStakes` to skip those vaults. As a result, only vaults that are still staked will be considered. This means that users who unstake their vaults after the rewards distribution has begun will lose out on rewards, as unstaked vaults are excluded from the reward calculation.

RECOMMENDATION

Instead of calling `getUserVaultStakes` use `getVaultStake`

RESOLUTION

Status: Acknowledged.

HIGH **H-07**

RESOLVED

User Can Influence Distribution In ZapRewards

CATEGORY Logical Error
LOCATION ZapRewards.sol
REVIEW Main Review

DESCRIPTION

In `calculateDailyVaultHolderWeightingsForChonk`, the total standard pool weight for all users is calculated and stored. This assumes that when `saveCalculatedVaultHolderRewardsToState` is called, that the users' info has not been altered.

A malicious user could increase his share by purchasing more vaults in between this two-step process and would then earn more rewards at the expense of other users.

RECOMMENDATION

Consider locking updates to user info in between this two-step process.

RESOLUTION

Status: Resolved.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/cd3ced611c30721121c459fed19e6647a4cf8b62>

HIGH **H-08**

RESOLVED

Incorrect Forecasted Rewards

CATEGORY	Logical Error
LOCATION	ZapRewards.sol: 329 – 336
REVIEW	Main Review

DESCRIPTION

The protocols offers users the option to stake their vaults, so they are awarded rewards not just for their current emitted tokens but also on all of their forecasted emissions for the duration of their stake.

The `postHalvingEventEmissions` are calculated by forecasting the rewards over the stake duration past the next halving, using the total daily rewards at next halving event as the daily emission rate. The issue is that some stakes can cross more than a single halving period.

Consider the following example:

- The current day is 179, `HALVING_EVENT_1_OCCURANCE_DAY` is at 180 days, and `HALVING_EVENT_2_OCCURANCE_DAY` is at 360 days.
- User stakes for 360 days, which will pass `HALVING_EVENT_1_OCCURANCE_DAY` and `HALVING_EVENT_2_OCCURANCE_DAY`.
- When calculating the user's `postHalvingEventEmissions` prior to the first halving, `tokensVestedAtNextHalvingPeriod` will be `HALVING_EVENT_1_DISTRO_PER_DAY`
- `postHalvingEventEmissions` are inflated for the user since the `HALVING_EVENT_2_DISTRO_PER_DAY` is ignored for a portion of their stake duration.
- 1 day accounted for the current emission until the first halving event, 180 days accounted for the emissions after the first halving, and the 179 days of vesting after the second halving event ended up using the daily emissions from the first halving event.

Ultimately, the `totalTokensToVestOverStake` is inflated and certain users will earn more rewards at the expense of others. Also note that the `dailyTokenEmissionPerVault` in the calculation is assumed to be consistent across the days staked, but the daily token emission can change from day-to-day as the proportion of vaults a user holds relative to total vaults sold changes.

RECOMMENDATION

H-08 Incorrect Forecasted Rewards

CONTINUED

Consider adding logic to account for multiple halvings being passed. Furthermore, document the assumption about the daily emission rate staying constant for the forecast.

— RESOLUTION

Status: Resolved.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/89c35197c119b8274595021cd31324bcccc2ca86>

HIGH **H-09**

RESOLVED

existingLockedTokensForStake Wrong Denominator

CATEGORY Logical Error
LOCATION ZapRewards.sol: 340
REVIEW Main Review

DESCRIPTION

In ZapRewards, the `_calculateEligibleStandardPoolTokensForUser` function calculates locked rewards that have already vested out as:

```
uint256 existingLockedTokensForStake = (totalUserLockedTokens * vaultsStaked) /  
totalVaultsHeldByUser
```

However, `totalVaultsHeldByUser` includes claimed vaults when it should only be using unclaimed vaults.

Consider this example:

Alice has claimed 2 out of 5 vaults and vested 100 tokens for the day

tokensUnlocked: $2/5 * 100 = 40$

tokensLocked: $3/5 * 100 = 60$

If she has staked all remaining 3 vaults:

```
existingLockedTokensForStake :  $3/5 * 60 = 36$ 
```

This implies that she never gets the full rewards for her locked tokens despite staking all remaining vaults. Instead, she should benefit from the full locked amount.

RECOMMENDATION

Change the calculation to:

```
(totalUserLockedTokens * vaultsStaked) / (totalVaultsHeldByUser - userState.vaultsClaimed)
```

RESOLUTION

Status: Resolved.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/730c091520c398f985497c05501dee16f0f38326>

HIGH **H-10**

RESOLVED

Not earning rewards after vesting ends

CATEGORY DOS
LOCATION Global
REVIEW Main Review

DESCRIPTION

After vesting ends, stakers should still receive rewards in `ZapRewards`. This should be done by first calling `calculateDailyVaultHolderWeightingsForChonk` in order to `_resetState`. However, that function will invoke `VaultVesting.getDaysLeftOfVesting()` which will revert because `daysCalculated > TOTAL_DAYS_VESTING`.

RECOMMENDATION

Return 0 from `getDaysLeftOfVesting` if `daysCalculated > TOTAL_DAYS_VESTING`.

RESOLUTION

Status: Resolved.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/abbbd3941153ec8a88412bcd5e35d3b91685e228>

HIGH **H-11**

RESOLVED

Continue over require in save

CATEGORY DOS
LOCATION ZapRewards::215
REVIEW Main Review

— DESCRIPTION

When rewards are saved for vaults, the transaction will revert if `standardPoolTokensEligibleForRewards` is 0 because of the following `[require]`(<https://github.com/GuardianAudits/zap-2/blob/50066b0e633268267631ec8643b752494b5126f9/contracts/ZapRewards.sol#L215>) statement. A user may (maliciously or not) cause a DOS with this revert if all of their stakes have expired. The `[getVaultStakerById]`(<https://github.com/GuardianAudits/zap-2/blob/50066b0e633268267631ec8643b752494b5126f9/contracts/ZapStaking.sol#L83-L86>) will return the address of the staker even if the stake is expired. Then `_checkIfShouldProcessReward()` will return `false` and `standardPoolTokensEligibleForRewards` will end up being 0 causing the whole transaction to revert.

Another possible case when this may happen is if after the vesting period ended a user with no locked tokens stakes a new vault.

— RECOMMENDATION

Consider doing a `continue` instead of `require` revert.

— RESOLUTION

Status: Resolved.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/2b7932b8d675b0c5f9e6c56e7e32722181a89c83>

HIGH **H-12**

RESOLVED

Early Unstake Fee Calculation Could Underflow

CATEGORY DOS
LOCATION ZapStaking.sol
REVIEW Main Review

DESCRIPTION

When users unstake early `calculateEarlyUnstakeFee` applies a fee % they have to pay based on how much of their stake duration has passed. This function doesn't consider stakes with `autoRelock = true`.

Because of that, after a stake relocks, `percentageProgressThroughStake` will exceed 100%. In result:

- For as long as this percentage is less than 100% of the 75% of the stake duration, `autoRelock` users will be able to unlock earlier by paying much smaller fee.
- Once the percentage goes above 100% of these 75% of the stake duration, the following calculation will revert and `autoRelocks` will not be able to unstake:

```
(100 * PCT_BASE) - percentageProgressThroughStake
```

RECOMMENDATION

Add a modulus operator in `calculateEarlyUnstakeFee` to handle `autoRelocks`

RESOLUTION

Status: Resolved.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/ff22c060e6d969fea777b0342bedd4b7b69b7228>

MEDIUM **M-01**

RESOLVED

Missing Recovery Functions

CATEGORY Logical Error
LOCATION VaultVesting.sol
REVIEW Main Review

DESCRIPTION

A large amount of ZapTokens are expected to be transferred to the VaultVesting and ZapRewards contracts for users to claim.

After the entire vesting duration, it is likely that there will be leftover tokens unclaimed. Furthermore, there may be some leftover tokens due to rounding over time. These tokens should be recoverable by admin.

RECOMMENDATION

Consider allowing ZAP tokens to be covered by privileged address post-vesting period.

RESOLUTION

Status: Resolved.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/83b36e7287d1e0b1ad76b48216406b555d51a350>

MEDIUM **M-02**

ACKNOWLEDGED

Less rewards for expired stakes

CATEGORY Logical Error
LOCATION ZapRewards
REVIEW Main Review

DESCRIPTION

If a user unstakes after the `rewardsDistroStartTimestamp`, they should still get staking rewards. However, if the stake has expired after that timestamp, the user will lose their rewards.

This is because `_checkIfShouldProcessStakeForReward` will mark the stake as invalid during the calculations. This is asymmetrical behavior that may lead to unexpected loss of rewards for some users.

RECOMMENDATION

Consider checking if the stake has expired after the start distribution timestamp and if so, reward them accordingly.

```
- if (!autoRelock && expiresAt < block.timestamp) return (false, amountStaked);  
+ if (!autoRelock && expiresAt < rewardsDistroStartTimestamp) return (false, amountStaked);
```

RESOLUTION

Status: Acknowledged.

MEDIUM **M-03**

RESOLVED

Off by One Error In vestingPeriodHasNotElapsed

CATEGORY Off By One
LOCATION VaultVesting.sol: 89-91
REVIEW Main Review

DESCRIPTION

`vestingPeriodHasNotElapsed` modifier will pass when the `daysCalculated` is 900. However, it should revert when calculated days reaches to 900.

RECOMMENDATION

Change the modifier to `require(TOTAL_VESTING_DAYS > daysCalculated)`

RESOLUTION

Status: Resolved.

<https://github.com/Lithium-Ventures/zap-node-sale/pull/6/commits/b640e23780369933c4b77015dfd1c18eb93086db>

MEDIUM **M-04**

RESOLVED

Restaking Can Be Gamed For Higher Rewards

CATEGORY Logical Error
LOCATION ZapStaking.sol
REVIEW Main Review

— DESCRIPTION

While enabling auto-relock may seem like the optimal strategy for maximizing staking rewards, savvy users can achieve better results by manually restaking daily.

By doing this, they ensure their remaining staking days are consistently maximized, allowing them to earn more rewards than users who rely solely on auto-relock.

This behavior is unexpected and may create an unfair advantage for users who manually restake, potentially disadvantaging regular users who choose the auto-relock option.

— RECOMMENDATION

Consider if this gaming is acceptable to the protocol. If not, a potential solution is to allow restaking only after some time has passed. Alternatively, a redesign of the forecasted rewards mechanism may be required.

— RESOLUTION

Status: Resolved.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/93933817319763a9730227235c09b74d537d1ab4>

LOW **L-01**

RESOLVED

Inconsistency Between startingIndexOfChonks

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

Both VaultVesting and ZapRewards use a variable startingIndexOfChonk when calculating daily weights in calculateTotalBonusWeightingForChonk and calculateDailyVaultHolderWeightingsForChonk respectively.

However, VaultVesting expects startingIndexOfChonk to be 1 -- since the first user index starts from 1 -- while ZapRewards expects it to be 0 since it does address vaultStaker = zapStaking.getVaultStakerByIndex(i + 1); to account for user index also starting from 1.

This discrepancy can lead to admin errors and possibly result in missing the first user when calculating daily weights in ZapRewards.

— RECOMMENDATION

Be aware of this difference and account for it in offchan processes.

— RESOLUTION

Status: Resolved.

LOW **L-02**

RESOLVED

Gas Wastage In Loop

CATEGORY Gas Optimization
LOCATION ZapRewards.sol: 268
REVIEW Main Review

— DESCRIPTION

The loop in `calculateDailyVaultHolderWeightingsForChonk` calls `_getCurrentVestingInformation` for each user. However, this information does not change across users and could only be called once.

— RECOMMENDATION

Consider retrieving the vesting information outside of the for-loop.

— RESOLUTION

Status: Resolved.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/618af21668467675e33f03ae1d704d63a55903a5>

LOW **L-03**

ACKNOWLEDGED

Wrong VaultVesting iterations count

CATEGORY	Logical Error
LOCATION	VaultVesting
REVIEW	Main Review

— DESCRIPTION

Function `calculateTotalBonusWeightingForChonk` tries sets a limit of how much iterations can be done. If the starting index of the chonk and the chonk size surpass the available accounts, the result is capped. However, the following calculation is wrong:

```
amountOfLoopIterations = (nextAccountID - startingIndexOfChonk) + 2
```

Let's say there are 2 master accounts and 2 normal accounts. If admins start from 0 and pass a very large chonk size, we will be calculating this amount of loop iterations as:

```
amountOfLoopIterations = (5 - 0) + 2 = 7
```

This will result in looping 7 times and calculating bonus weights for accounts that don't exist, i.e `address(0)`.

— RECOMMENDATION

Don't add 2 to `amountOfLoopIterations`

— RESOLUTION

Status: Acknowledged.

LOW **L-04**

RESOLVED

Uninitialized Treasury Account In ZapStaking

CATEGORY	Warning
LOCATION	ZapStaking.sol
REVIEW	Main Review

DESCRIPTION

The treasury account is not set in `initialize` but within `setTreasuryAddress` by an admin.

While the treasury address is the zero address, unstake functions would revert when trying to send tokens to the zero address.

RECOMMENDATION

Be aware of this and set the treasury address early, potentially in the `initialize` function.

RESOLUTION

Status: Resolved.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/ef0147f46f14c558f0d01c0702f948288e1d6cf1>

LOW **L-05**

RESOLVED

Unused State Variables

CATEGORY	Superfluous Code
LOCATION	ZapStaking.sol: 24, 39
REVIEW	Main Review

DESCRIPTION

`nextTokenStakerId` variable and `allTokenStakers` mapping in the `ZapStaking` contract are never used.

RECOMMENDATION

Consider removing unused state variables.

RESOLUTION

Status: Resolved.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/ef1b7a7177a715738da3a31465df941704d1bd32>

LOW **L-06**

RESOLVED

Incorrect Parameter In Unstaked Event

CATEGORY Events
LOCATION ZapStaking.sol: 286, 318
REVIEW Main Review

DESCRIPTION

The `Unstaked` event includes an `earlyUnstakeFee` parameter which is hardcoded to 0 when emitting events, regardless of whether the fee was actually 0 or not.

RECOMMENDATION

It is recommended to emit the actual early unstake fee during events.

RESOLUTION

Status: Resolved.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/b8b023dbfffc93a62433f534b237c6160a8c4368e>

LOW **L-07**

ACKNOWLEDGED

Forecast Doublecounting Current Day

CATEGORY Warning
LOCATION ZapRewards
REVIEW Main Review

— DESCRIPTION

The total tokens over the lifetime of the vault stake is calculated with `uint256`
`totalTokensOverLifetimeOfStake = totalTokensToVestOverStake + existingLockedTokensForStake;`

If the calculation was done at the exact time such that `vaultStake.expiresAt - timeNow` is a perfect multiple of days, it is possible for `totalTokensToVestOverStake` to include the current day as remaining.

This can be problematic since `existingLockedTokensForStake` already included the rewards for the current day, which can lead to double counting and inflated standard pool rewards.

— RECOMMENDATION

This is an unlikely scenario, so just be aware of when calculations are performed.

— RESOLUTION

Status: Acknowledged.

LOW **L-08**

RESOLVED

Updating Wrong Variable For Token Staker Rewards

CATEGORY Warning
LOCATION ZapRewards.sol: 96
REVIEW Main Review

DESCRIPTION

In `calculateTotalTokenStakerRewardsForChonk`, the loop updates the state variable `totalStandardPoolWeighting` instead of the local variable `totalStandardPoolTokens`.

This results in gas wastage and defeats the purpose of the local variable.

RECOMMENDATION

Increment the `totalStandardPoolTokens` variable inside the loop.

RESOLUTION

Status: Resolved.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/09b881696b905abc8bde456ff4bc8e5afe819f71>

LOW **L-09**

ACKNOWLEDGED

Vault staking incentives

CATEGORY Warning
LOCATION Global
REVIEW Main Review

— DESCRIPTION

If a user got 0 locked tokens and acquires a new vault that they stake it after the vesting has ended they will receive no rewards.

— RECOMMENDATION

Warn users they have no benefit of staking in these conditions.

— RESOLUTION

Status: Acknowledged.

LOW **L-10**

RESOLVED

Multiply Before Dividing

CATEGORY Precision
LOCATION ZapRewards.sol: 133
REVIEW Main Review

DESCRIPTION

In `_calculateTokensWithNegativeMultiplier`, tokens are divided by `100` before multiplying by `negativeMultiplier`.

This leads to loss of precision due to rounding and users therefore lose tokens. For example, even if `negativeMultiplier` was `100`, the user will still experience some loss due to rounding.

RECOMMENDATION

Perform multiplication before the division.

RESOLUTION

Status: Resolved.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/509fa3c9042f08b97f9675792005d5ed77efe91c>

LOW **L-11**

ACKNOWLEDGED

Daily function may be executed more

CATEGORY Validation
LOCATION Global
REVIEW Main Review

— DESCRIPTION

There are numerous daily functions which have to be called every day in order to calculate state variables used for rewarding both in `VaultVesting` and `ZapRewards`. Currently, these function can be called more than once per day. If that happens, the state variables used for calculating rewards will be incorrect and users will receive wrong rewards.

— RECOMMENDATION

Be sure to call these function exactly once per day.

— RESOLUTION

Status: Acknowledged.

LOW **L-12**

RESOLVED

Incorrect Tokens Emitted On Certain Days

CATEGORY Logical Error
LOCATION VaultVesting.sol: 358
REVIEW Main Review

DESCRIPTION

In `calculateDailyVestingForChonk`, `daysCalculated` is incremented before calculating `tokensEmitted`. This inflates `tokensEmitted` if `daysCalculated` crosses into a new halving event.

RECOMMENDATION

Calculate `tokensEmitted` first before incrementing `daysCalculated`.

RESOLUTION

Status: Resolved.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/999a993aadee608586980ff9c2d63f2f6e8c71b9>

LOW **L-13**

RESOLVED

preTGE ratio is the same for all users

CATEGORY	Warning
LOCATION	ZapRewards
REVIEW	Main Review

— DESCRIPTION

Everyday stakers receive rewards from the `ZapRewards` standard pool based on their `(userWeight / totalWeight)` proportion. There are also bonus pool rewards given to vault stakers and these rewards should be calculated using the value obtained by the user's preTGE tokens multiplied by a similar ratio.

```
//In saveCalculatedVaultHolderRewardsToState
standardPoolRewards = (standardPoolTokensEligibleForRewards * DAILY_STANDARD_POOL_TOKENS) /
totalStandardPoolWeighting
// In _calculateBonusPoolTokensForVaultHolder
bonusTokens = standardPoolRewards * preTGE / standardPoolTokensEligibleForRewards=>
bonusTokens = (standardPoolTokensEligibleForRewards * DAILY_STANDARD_POOL_TOKENS /
totalStandardPoolWeighting) * preTGE / standardPoolTokensEligibleForRewards=>
bonusTokens = (standardPoolTokensEligibleForRewards * DAILY_STANDARD_POOL_TOKENS * preTGE) /
(totalStandardPoolWeighting * standardTokensEligibleForRewards) =>
bonusTokens = (DAILY_STANDARD_POOL_TOKENS * preTGE) / totalStandardPoolWeighting
```

After unraveling the bonus pool tokens earned, the final proportion used to adjust the staked pre-TGE tokens is the `(DAILY_STANDARD_POOL_TOKENS / totalStandardPoolWeighting)`. The amount of tokens in the standard pool is hardcoded to `12950e18` and the `totalStandardPoolWeighting` is shared across all users in the calculation. Consequently, the only unique portion of the bonus pool rewards for each user is the amount of pre-TGE tokens they hold per staked vault. This may be unexpected as the pre-TGE tokens are not directly correlated with the standard pool weighting, which is contrast to the standard pool reward calculation.

— RECOMMENDATION

Ensure this is the intended protocol tokenomics for bonus pool rewards.

L-13 preTGE ratio is the same for all users

CONTINUED

— RESOLUTION

Status: Resolved.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/c4b238a64c933e04d9ce4a968ec4a2ec8c1a3a78>

LOW **L-14**

RESOLVED

Potentially Unexpected Returning Staker Calculation

CATEGORY Logical Error
LOCATION ZapStaking.sol: 110-122
REVIEW Main Review

DESCRIPTION

In functions `isReturningVaultStaker` and `isReturningTokenStaker` the `userStakeCount` is incremented regardless of the status of the user's stake. Because it will always be incremented to three since there are three tiers, a vault staker will only be considered returning if a user had previously unstaked from all three tiers which may be unexpected functionality.

RECOMMENDATION

Clarify the logic behind returning stakers or just set `userStakeCount = 3` if the function is meant to check whether a user unstaked from all three vaults previously.

RESOLUTION

Status: Resolved.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/9eff9c491a64c7b80112f8ed5d1fc1a87e77d540>

LOW **L-15**

RESOLVED

Import Can Be Finished Multiple Times

CATEGORY Validation
LOCATION VaultVesting.sol: 280
REVIEW Main Review

DESCRIPTION

The default admin is able to call function `finishImport` multiple times, which can be used to change `daysCalculated` to various values and negatively influence what halving the protocol is currently in and how many rewards are distributed.

RECOMMENDATION

Consider restricting how many times `finishImport` can be called or clearly documenting this behavior.

RESOLUTION

Status: Resolved.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/5347b36f59819d329aa107d5409c168b18089d1d>

CRITICAL **C-01**

ACKNOWLEDGED

Underflow

`_createNewExpiryTimeForAutoRelock()`

CATEGORY —

LOCATION ZapStaking::L448

REVIEW Remediation Review

DESCRIPTION

`_createNewExpiryTimeForAutoRelock()` checks if `expiresAt` is less than `timeNow`, and returns early if that is the case. Otherwise it will attempt to store `timeNow` minus `expiresAt` into a `uint256`. For this line of code to be hit, `expiresAt` must be greater than or equal to `timeNow`, which will cause an underflow. This will prevent users with `autoRelock` enabled from restaking early.

RECOMMENDATION

Change the if statement to check if `expiresAt` is larger than `timeNow`.

RESOLUTION

Status: Acknowledged.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/4da191dc11e04cbf179beec40324d3a8994d88dd>

HIGH **H-01**

ACKNOWLEDGED

Wrong standardPoolNegativeMultiplier Applied

CATEGORY —
LOCATION ZapRewards::L108
REVIEW Remediation Review

DESCRIPTION

Similarly to `C-04` from the main review, `_calculateTotalTokenStakerTokens` uses the wrong tier for calculations. However, this time it is using the wrong `standardPoolNegativeMultiplier` when making the calculation for `userTokenWeighting`.

This occurs because `_calculateTotalTokenStakerTokens()` loops through `tokenStakes` and uses `i` to get the multiplier from `stakingTiers`. If a user only stakes in the third tier, `i` will be 0 and it will use the negative multiplier from the first tier.

RECOMMENDATION

Loop through each tier and if the user does not have any tokens staked in that tier then `continue`.

RESOLUTION

Status: Acknowledged.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/241f15ddc8bfc88e6cb12de44b2458a3e556efe0>

MEDIUM **M-01**

ACKNOWLEDGED

Wrong halfway time calculation

CATEGORY —
LOCATION ZapStaking.restakeVaults
REVIEW Remediation Review

DESCRIPTION

`ZapStaking.restakeVaults()` should allow restaking of vaults only if half of the stake's duration has passed. However, the `expiresAt` variable it passes to the `_isLessThanHalfWayThroughStake()` function is the time the stake would expire after the restake. In result, the duration of the stake will be greater than it should be and users won't be able to restake their vaults if they haven't passed halfway that duration.

RECOMMENDATION

Instead of `expiresAt`, pass `vaultStakes[msg.sender][tierIndex].expiresAt` to `_isLessThanHalfWayThroughStake()`

RESOLUTION

Status: Acknowledged.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/53afec7bc82e562f7ed77d66c56d69d9db040de7>

MEDIUM **M-02**

ACKNOWLEDGED

Halfway calculations ignore autoRelock

CATEGORY —
LOCATION ZapStaking
REVIEW Remediation Review

DESCRIPTION

The `_isLessThanHalfWayThroughStake()` function will return false if `timeNow > expiresAt`. This is fine for normal stakes, but ones with `autoRelock = true` will not yield correct result. Once after the first relock happens, the above condition will always be met and users will be able to restake vaults whenever they want to.

RECOMMENDATION

Consider adding `autoRelock` logic to `_isLessThanHalfWayThroughStake()`.

RESOLUTION

Status: Acknowledged.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/53afec7bc82e562f7ed77d66c56d69d9db040de7>

MEDIUM **M-03**

ACKNOWLEDGED

Wrong autoRelock unstake fee

CATEGORY —
LOCATION ZapStaking
REVIEW Remediation Review

DESCRIPTION

In order to calculate the fee that needs to be paid when early unstaking an `autorelock` stake, the `expiresAt` of that stake is adjusted to match the end time of the current (relocked) stake.

Then that new `expiresAt` is used in `calculateEarlyUnstakeFee`. However, when such a state relocks, it `stakedAt` should also be updated. Because that's not the case, the calculations will be wrong and users will pay less fee because their percentage through the stake will be calculated higher.

Let's look at an example:

- Stake on day 0 for 180 days with `autoRelock = true`
- We are on day 200, which is 20/180 of the stake
- The new `expiresAt` will be 360
- The elapsed time is $200 - \text{stakedAt} = 200$
- Duration = 360
- Calculations will return $200 / 360$ which is a lot bigger than 20/18

RECOMMENDATION

Adjust the `stakedAt` for `autoRelock` as well.

RESOLUTION

Status: Acknowledged.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/dfef74ce86f4beb4211fc4b5b2f2b48b6c975ba9>

MEDIUM **M-04**

ACKNOWLEDGED

recoverTokensAfterVesting off by one

CATEGORY —
LOCATION VaultVesting
REVIEW Remediation Review

DESCRIPTION

Vesting is completed when `daysCalculated` is equal to or greater than `TOTAL_VESTING_DAYS` but the current recovery mechanism will prevent token recovery when `TOTAL_VESTING_DAYS == daysCalculated`:

```
require(TOTAL_VESTING_DAYS < daysCalculated, "Vesting period has not completed");
```

RECOMMENDATION

Modify `<` to `<=` in `recoverTokensAfterVesting`

RESOLUTION

Status: Acknowledged.

<https://github.com/Lithium-Ventures/zap-node-sale/commit/3e48b99c5bdb07e049312c90ba7a40d6544fc4c7>

LOW **L-01**

ACKNOWLEDGED

Malicious token recovery

CATEGORY —
LOCATION VaultVesting
REVIEW Remediation Review

DESCRIPTION

The newly added `VaultVesting.recoverTokensAfterVesting()` function allows the admin of the contract to recover any tokens sent to the contract. The intention of that function is to retrieve the difference between the originally sent tokens and the actually distributed amount. However, this function can be used by the admin to transfer locked/unlocked tokens of eligible users out of the contract.

RECOMMENDATION

Be aware of the risk.

RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Unstaking fees affects rewards

CATEGORY —
LOCATION Global
REVIEW Remediation Review

DESCRIPTION

When users unstake their vaults, the fees paid are taken from their locked tokens. This will affect the rewards earned in `ZapRewards` for the past because locked tokens will have decreased, but the available vaults will not change. Because of that users will receive overall less rewards.

RECOMMENDATION

Be sure you are aware of this behavior.

RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.